

CIP-A104 Practical Laboratories — Lab 5

OWASP ZAP Integrated Assessment, Remediation and Retest Capstone

Student: Janet Michael

Lab: CIP-A104 Lab 5

Date: 6 September 2026

Environment: Kali Linux VM

Targets: DVWA and OWASP Mutillidae II

Tools: OWASP ZAP, Browser Developer Tools, curl

Executive Summary

This assessment was performed against two intentionally vulnerable applications, DVWA and Mutillidae II, running locally in an isolated lab environment.

OWASP ZAP was used for passive assessment, site discovery and active scanning, while manual testing was used to confirm selected findings.

The most important confirmed finding was **Reflected XSS in DVWA**, where a harmless JavaScript payload executed in the browser. Weak session-cookie security was also observed in DVWA, while Mutillidae exposed server, application and database-related information.

ZAP also reported other high-risk alerts, including XSS, SQL Injection, Path Traversal and Remote File Inclusion in Mutillidae. These were not reported as confirmed vulnerabilities because they were not manually reproduced.

Remediation testing showed improved behavior for the tested XSS cases at the stronger security level. Both applications were subsequently reset to their clean state.

1. Introduction

This practical was carried out to perform an end-to-end security assessment of two intentionally vulnerable web applications, **Damn Vulnerable Web Application (DVWA)** and **OWASP Mutillidae II**.

The assessment combined automated security testing using **OWASP ZAP** with manual testing. The purpose was to identify possible vulnerabilities, review the alerts produced by ZAP, manually validate selected findings, determine their impact, apply security improvements and perform retesting.

The applications were running locally inside the lab environment. This kept the testing within the authorized scope and prevented testing from affecting public or unrelated systems.

2. Objectives

The objectives of this lab were to:

1. Define the scope and rules for the security assessment.
2. Configure OWASP ZAP for the authorized local applications.
3. Browse DVWA and Mutillidae through ZAP and perform passive assessment.
4. Review the site trees and alerts generated by ZAP.
5. Perform constrained automated scanning against the authorized applications.
6. Manually inspect selected ZAP findings.
7. Confirm vulnerabilities using minimal-impact testing.
8. Separate confirmed findings from alerts that could not be manually confirmed.
9. Determine the observed and possible impact of the vulnerabilities.
10. Apply stronger security settings and perform remediation retesting.
11. Prioritize the validated findings.
12. Reset the applications and clean up the testing environment.
13. Prepare an evidence-based security assessment report.

3. Lab Environment

The assessment was performed using a **Kali Linux virtual machine**.

The two vulnerable applications were hosted locally using Docker.

DVWA

DVWA was accessed using: <http://127.0.0.1:4280>

OWASP Mutillidae II

Mutillidae II was accessed using: **“http://127.0.0.1:8888”**

The main tools used during the assessment were:

- OWASP ZAP
- Web browser
- Browser developer tools
- curl
- Docker
- Kali Linux

A separate workspace was also prepared for storing the evidence and notes: **“~/CIP-A104-Lab5”**

The workspace contained folders for:

1. Evidence
2. Requests
3. Responses
4. ZAP
5. Notes
6. Screenshots
7. Report

An activity log and evidence register were also created in the Notes directory.

4. Scope and Rules of Engagement

Before beginning the assessment, the testing scope was limited to the two local vulnerable applications.

In-scope targets

DVWA:

<http://127.0.0.1:4280>

Mutillidae II:

<http://127.0.0.1:8888>

Only these applications were included in the assessment.

The testing was performed against the local lab environment and not against public websites or external systems.

Testing limitations

The following activities were not performed:

- Database dumping
- Credential dumping
- Privilege escalation
- Persistence
- Lateral movement
- Destructive testing
- External callbacks
- Testing unrelated systems
- Cookie theft
- Session hijacking
- Accessing unrelated data

For XSS validation, harmless JavaScript alert markers were used instead of any payload designed to steal information.

The assessment was also stopped where further testing would have required unnecessary or intrusive exploitation.

6. OWASP ZAP Configuration

OWASP ZAP was used as the main security testing tool for the assessment.

After starting ZAP, I enabled **Protected Mode**. This was important because the assessment was restricted to the applications that had been deliberately added to the testing scope.

Both applications were added to the ZAP scope.

The DVWA target was: “ <http://127.0.0.1:4280>”

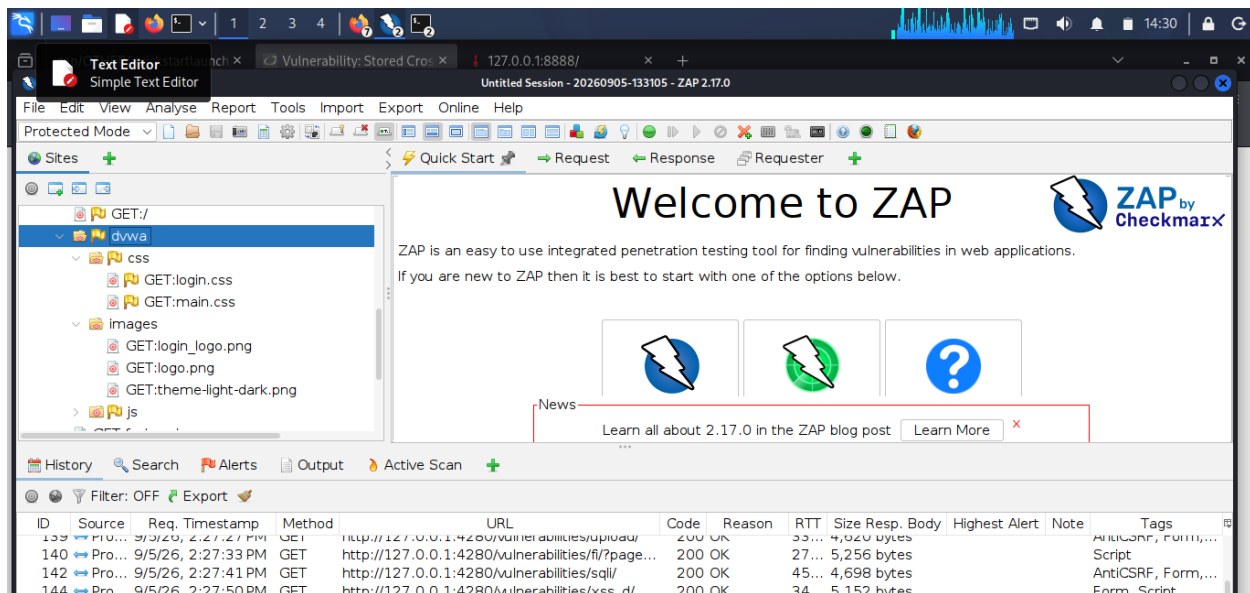
The Mutillidae target was: “ <http://127.0.0.1:8888>”

The contexts used for the applications were based on:

http://127.0.0.1:4280.*

http://127.0.0.1:8888.*

This configuration helped ensure that ZAP's assessment was limited to the local lab applications.



7. Passive Assessment

After configuring ZAP, I browsed through both applications while ZAP monitored the traffic.

The purpose of passive scanning was to identify possible security issues from the requests and responses without deliberately modifying the application traffic.

The alerts generated by passive scanning were then reviewed.

Not every alert was automatically treated as a confirmed vulnerability. Alerts were considered possible findings until they were reviewed against the available evidence.

7.1 DVWA Passive Scan Results

During the DVWA assessment, ZAP identified several alerts.

The alerts included:

Medium-risk alerts

- Absence of Anti-CSRF Tokens
- CSP: Failure to Define Directive with No Fallback
- CSP: Wildcard Directives
- CSP: Style Source Unsafe Inline
- Content Security Policy Header Not Set
- Cross-Domain Misconfiguration
- HTTP to HTTPS Insecure Transition in Form Post
- Missing Anti-clickjacking Header

Low-risk alerts

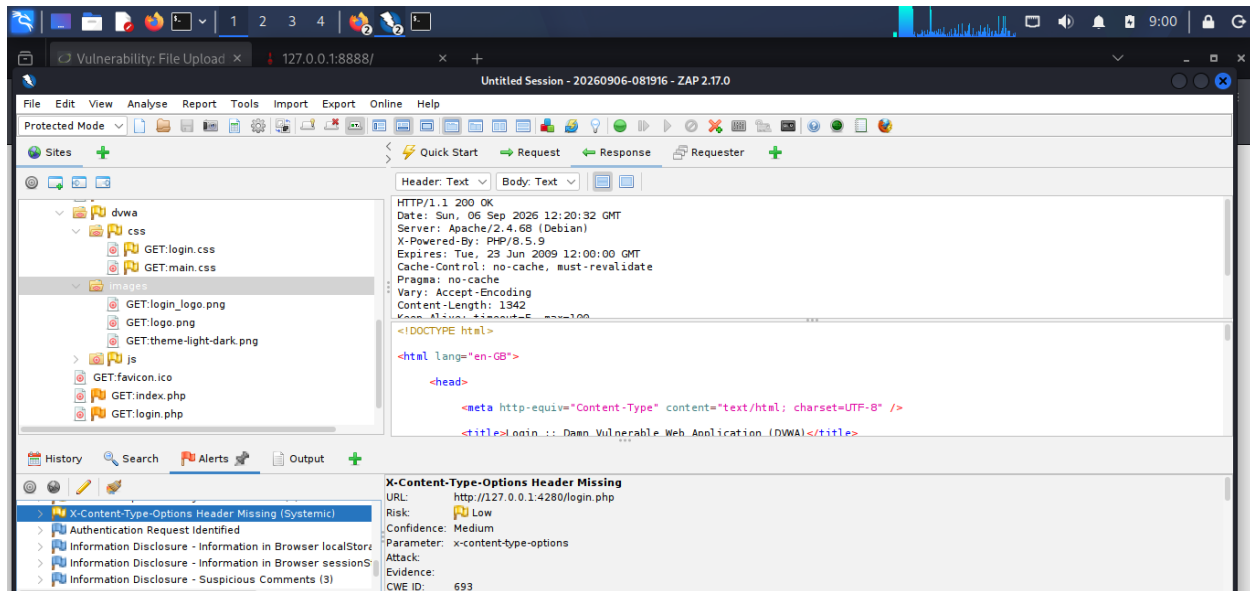
- Sub-Resource Integrity Attributes Missing
- Vulnerable JavaScript Library
- Cookie No HttpOnly Flag
- Cookie Without SameSite Attribute
- Cross-Domain JavaScript Source File Inclusion
- Server Leaking Information via X-Powered-By HTTP Response
- Server Leaking Version Information via Server HTTP Response
- Strict Transport Security Header Not Set

Informational alerts

ZAP also identified informational results such as:

- X-Content-Type-Options Header Missing
- Authentication Request Identified
- Information Disclosure: Information in Browser Local Storage
- Information Disclosure: Information in Browser Session Storage

- Information Disclosure: Suspicious Comment
- Modern Web Application



These results were recorded as part of the passive assessment.

The passive scan was useful because it identified several security configuration issues that could then be reviewed in more detail.

7.2 Important DVWA Passive Findings

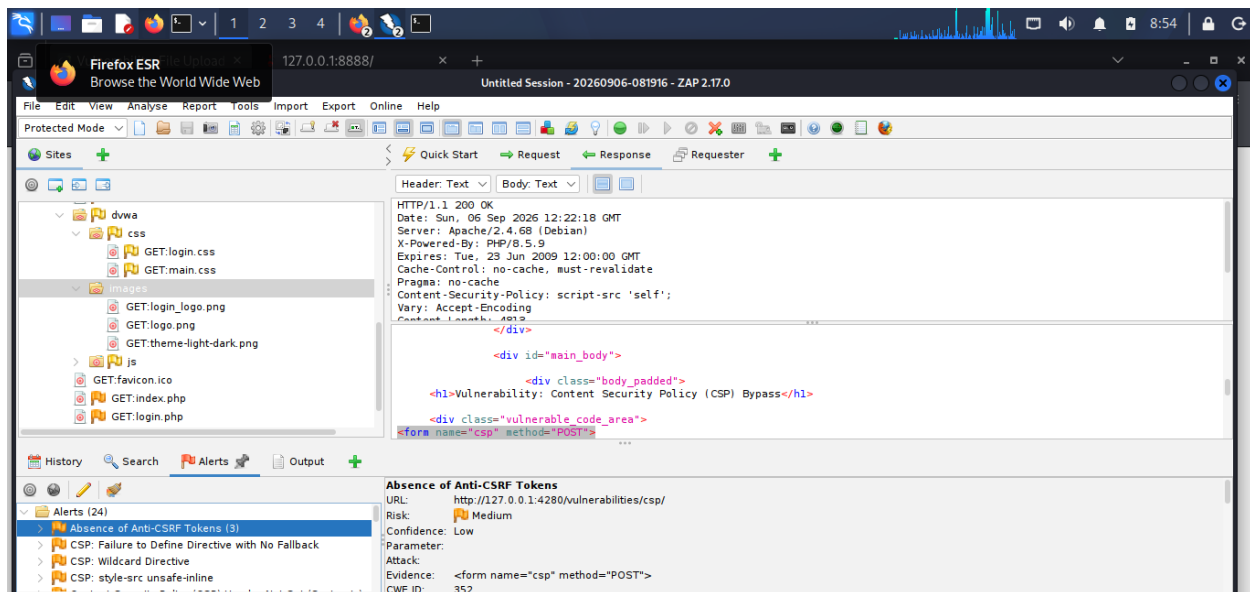
Some of the alerts received more attention because they related to security controls that were directly visible in the application responses.

For example, ZAP identified cookie security issues involving the HttpOnly and SameSite attributes.

ZAP also identified several Content Security Policy issues, including:

- CSP wildcard directives
- Unsafe inline style sources
- Missing or incomplete CSP protection

These alerts were considered during the manual assessment and final analysis.



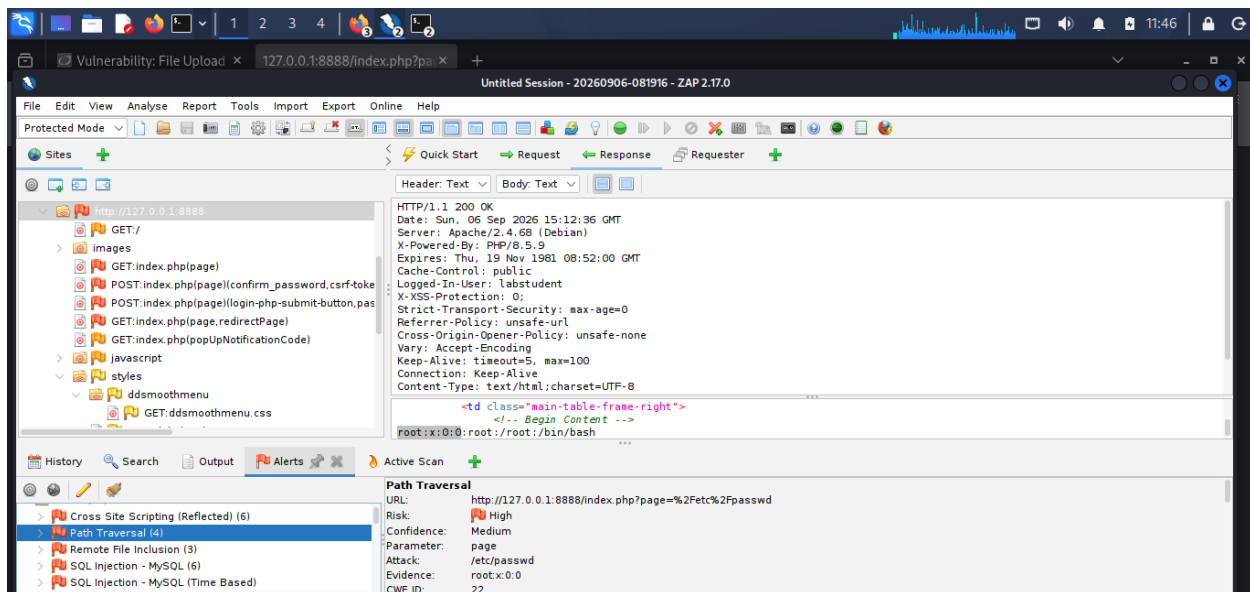
8. Mutillidae Assessment

Mutillidae II was also browsed through ZAP.

ZAP built a site tree for the application and identified a number of potential security issues.

The active scan results included the following:

Alert	Risk	Confidence
Cross-Site Scripting	High	Medium
Path Traversal	High	Medium
Remote File Inclusion	High	Medium
SQL Injection	High	Medium
Absence of Anti-CSRF Tokens	Medium	Low



These alerts were treated as **automated discoveries requiring validation**.

I did not manually exploit the SQL Injection, Path Traversal or Remote File Inclusion findings because the assessment was intended to use minimal-impact testing and did not require database extraction or other intrusive exploitation.

I attempted to manually investigate the Mutillidae XSS alert through the Echo Message functionality. However, the page did not load successfully.

ZAP reported: ‘**Failed to read <http://127.0.0.1:8888/index.php?page=echo.php> within 20 seconds**’

Because the page timed out, the Mutillidae XSS alert was **not recorded as manually confirmed**.

9. Constrained Automation and Active Scanning

After the passive assessment, OWASP ZAP was used to perform automated discovery and active scanning against the two applications.

The scanning was kept within the local lab environment and the targets that had already been placed in scope.

9.1 Site Discovery

ZAP was used to discover the pages and endpoints that were accessible from the two applications.

The site tree helped show the application structure and provided targets for further review.

The discovered content was reviewed before considering the alerts generated by the automated scan.

9.2 DVWA Automated Assessment

The DVWA application was included in the ZAP assessment and its discovered pages were monitored during testing.

ZAP identified several possible security weaknesses, including issues involving:

- Cross-Site Scripting-related behaviour
- Anti-CSRF protection
- Content Security Policy
- Cookie security
- Security headers
- Information disclosure
- JavaScript library security

The automated results were not treated as proof by themselves. Selected results were compared with the manual evidence collected during the assessment.

10. Alert Validation

After the automated scanning, selected alerts were reviewed using the available request, response and browser evidence.

The validation process was based on the following approach:

1. Review the ZAP alert.
2. Inspect the affected request or response.
3. Determine whether the reported security condition could be observed.
4. Where appropriate, perform a harmless manual test.
5. Record the actual effect.
6. Record any limitation that prevented full confirmation.

This approach was used to avoid reporting every automated alert as a confirmed vulnerability.

10.1 Validated Findings

The following three findings were selected for the final assessment:

1. **DVWA Reflected XSS**
2. **DVWA Session Cookie Security Weakness**
3. **Mutillidae Information Disclosure**

These findings covered different security areas and involved both applications.

10.2 Findings Not Manually Confirmed

Some ZAP alerts were not included as confirmed findings because they were not manually validated.

These included the Mutillidae alerts for:

- Cross-Site Scripting
- Path Traversal
- Remote File Inclusion
- SQL Injection
- Absence of Anti-CSRF Tokens

The reason for not confirming these findings was that the assessment did not require intrusive exploitation and, in the case of the Mutillidae XSS page, the page used for validation timed out.

Therefore, these alerts were retained as **automated findings requiring further investigation**, rather than being presented as manually confirmed vulnerabilities.

15. Impact Mapping and Observed vs Potential Impact

After confirming the selected findings, I considered what each vulnerability could affect within the applications. I separated the impact that I actually observed during testing from the impact that could potentially occur if the vulnerability were abused further.

I kept the impact assessment within the lab environment. I did not attempt privilege escalation, lateral movement, persistence, access to unrelated systems, or access to other users' sensitive information.

15.1 Impact Matrix

Finding	Observed Impact	Potential Impact	Application/Process Affected	Scope of Potential Impact
Reflected XSS	JavaScript alert executed in the user's browser	An attacker could potentially execute unwanted JavaScript in a victim's browser	User browser and affected DVWA page	Users who access a maliciously crafted affected page/request
Weak Session Cookie Security	HttpOnly, Secure and SameSite protections were	Increased risk of session exposure or session-related	Authentication/session management	Potentially an affected user session

	not enabled; cookie remained visible after logout	attacks if combined with another weakness		
Mutillidae Information Disclosure	Server, PHP and database-related information was visible in responses/source	Attackers could use the information to improve reconnaissance and identify technologies or configuration weaknesses	Web application and supporting server	Information exposed to anyone who can access the affected application

15.2 Reflected XSS — Impact

The greatest directly observed impact from my manual testing was the reflected XSS finding in DVWA.

The actual effect I observed was JavaScript execution in the browser. The proof was limited to an alert box, so I did not demonstrate access to cookies, account information, or another user's data.

The potential impact is higher than the demonstrated impact because successful XSS can allow malicious JavaScript to run in the security context of the affected application. Depending on the application's other controls, this could potentially affect user actions or expose information available to the browser.

However, I did not test these additional effects. Therefore, the report records **JavaScript execution as the observed impact** and treats further account or data impact as **potential rather than confirmed**.

15.3 Weak Session Cookie Security — Impact

The session cookie inspection showed that several security attributes were not enabled.

The observed conditions were:

- HttpOnly = False
- Secure = False
- SameSite = None

The session identifier also appeared unchanged before and after login, based on the evidence collected.

The cookie was still visible after logout, but I did not perform a complete test to determine whether the old session could still be used.

The potential impact is that weak cookie protection can make session security more exposed if another vulnerability is available. For example, the lack of HttpOnly can increase the risk of JavaScript accessing the cookie when an XSS vulnerability exists.

I therefore consider the observed issue to be a **session security weakness**, but I do not claim successful session hijacking because that was not tested.

15.4 Mutillidae Information Disclosure — Impact

The Mutillidae application returned information about the web server and application technology.

For example, the response exposed:

Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.9

The application also contained database-related information in an HTML comment.

The observed impact was therefore **information disclosure**.

The potential impact is that an attacker could use this information during reconnaissance to understand the technologies being used and identify possible areas to investigate. Version information can sometimes help an attacker search for known weaknesses affecting a particular technology.

I did not use the disclosed database information to attempt database access. Therefore, database compromise was not observed and is not included as a confirmed impact.

15.5 Privilege and Blast Radius Assessment

I also considered the privilege level and possible blast radius of the confirmed findings.

Finding	Current Privilege Observed	Current Privilege Observed
Reflected XSS	Normal application/user browser context	Affected users and browser-side activity on the vulnerable page
Session Cookie Weakness	Normal authenticated session context	Potential compromise of an affected session if another weakness is combined with it
Information Disclosure	Normal unauthenticated/application response access	Reconnaissance information about the application and server

No privilege escalation was performed during the assessment.

No evidence was collected showing that any of the findings allowed access to the host operating system, Docker host, another application, or unrelated systems.

15.6 Observed Impact vs Potential Impact

A major point from the assessment was the need to avoid treating every possible consequence as something that had actually happened.

For example, the XSS test confirmed JavaScript execution, but I did not steal cookies or take over an account. Similarly, the Mutillidae information disclosure showed database-related information, but I did not use that information to access the database.

This distinction makes the findings more accurate because the report is based on what I was able to reproduce during the lab.

16. False-Positive and Informational Alert Analysis

ZAP generated several alerts during the assessment. I did not treat every alert as a confirmed vulnerability. I reviewed the alerts and considered the confidence level, the application behaviour, and whether I could manually reproduce the issue.

This was important because automated scanners can identify possible security problems that still need manual verification.

16.1 Mutillidae XSS Alert

ZAP reported:

Cross-Site Scripting — High Risk, Medium Confidence

I attempted to manually investigate the XSS alert using the Mutillidae Echo Message page. However, the page did not load successfully and ZAP reported a timeout:

Failed to read <http://127.0.0.1:8888/index.php?page=echo.php> within 20 seconds.

Because I could not successfully load the affected page and reproduce the behaviour, I did not classify the ZAP XSS alert as a manually confirmed finding.

I recorded it as **not manually confirmed** rather than calling it a false positive.

This distinction is important because the test failed due to the page timeout, not because I proved that the vulnerability did not exist.

16.2 Mutillidae Path Traversal Alert

ZAP reported:

Path Traversal — High Risk, Medium Confidence

The alert indicated that the application might be vulnerable to path traversal.

I did not perform an intrusive manual path traversal test. In particular, I did not attempt to read operating system files or access files outside the application.

Therefore, the alert remained an **automated finding that was not manually confirmed**.

I did not report successful file access because I had no evidence showing that such access occurred.

16.3 Mutillidae Remote File Inclusion Alert

ZAP reported:

Remote File Inclusion — High Risk, Medium Confidence

I did not manually exploit this alert because doing so could involve loading an external resource or performing actions outside the intended minimal-impact testing.

Since no successful manual proof was obtained, I did not classify Remote File Inclusion as a confirmed vulnerability in my final findings.

The alert was retained as an automated result requiring further controlled validation in a suitable test environment.

16.4 Mutillidae SQL Injection Alert

ZAP reported:

SQL Injection — High Risk, Medium Confidence

I did not attempt database extraction or other intrusive SQL injection testing.

The lab rules specifically prohibited database dumping and destructive activity. Because I did not obtain manual evidence proving SQL injection, I did not claim successful database compromise.

The alert was therefore treated as an **automated detection requiring further validation** rather than a confirmed database compromise.

16.5 Absence of Anti-CSRF Tokens

ZAP reported:

Absence of Anti-CSRF Tokens — Medium Risk, Low Confidence

This alert was also seen during the DVWA passive assessment.

The presence of an alert does not automatically prove that a successful cross-site request forgery attack can be performed. A complete CSRF assessment would require checking the affected state-changing requests and determining whether another control prevents unauthorized requests.

Because the alert had low confidence and I did not perform a full CSRF attack, I did not treat it as one of my three main confirmed findings.

16.6 Informational Alerts

ZAP also reported several informational alerts. Examples included:

- Authentication Request Identified
- Modern Web Application
- Information Disclosure: Information in Browser Local Storage
- Information Disclosure: Information in Browser Session Storage
- Information Disclosure: Suspicious Comment
- X-Content-Type-Options Header Missing

I did not automatically classify these informational alerts as vulnerabilities.

Some of them provide useful information about the application but do not by themselves demonstrate that an attacker can compromise the application.

For example, an authentication request being identified by ZAP simply shows that ZAP recognized an authentication-related request. It does not prove that authentication can be bypassed.

Similarly, identifying a modern web application is informational and is not a vulnerability by itself.

16.7 Alert Classification Summary

ZAP Alert	Risk	Confidence	My Classification	Reason
Mutillidae XSS	High	Medium	Not manually confirmed	Affected page timed out during manual investigation
Mutillidae Path Traversal	High	Medium	Not manually confirmed	No manual file access test performed
Mutillidae SQL Injection	High	Medium	Not manually confirmed	No database extraction or

				intrusive SQLi test performed
Absence of Anti-CSRF Tokens	Medium	Low	Not selected as confirmed	Low confidence and no complete CSRF proof
Authentication Request Identified	Informational		Informational	Scanner identification, not a demonstrated vulnerability
Modern Web Application	Informational		Informational	Application classification
Browser Local Storage Information	Informational		Informational	Information-level observation
Browser Session Storage Information	Informational		Reviewed	Could provide information but required context
X-Content-Type-Options Header Missing	Informational		Informational	Header weakness without direct exploit demonstration

16.8 Why Manual Validation Was Important

The automated scan helped me identify areas that required attention, but it did not replace manual testing.

The Mutillidae scan is a good example. ZAP identified several high-risk alerts, including XSS, Path Traversal, Remote File Inclusion and SQL Injection. However, I could not manually confirm all of them within the allowed testing conditions.

On the other hand, the DVWA reflected XSS was manually reproduced using a harmless payload, so I was able to classify it as a confirmed finding.

The final report therefore gives more weight to findings that I could actually reproduce and support with evidence.

This helped prevent me from overstating the results of the automated scan.

17. Remediation and Retesting

After identifying the vulnerabilities, I tested the applications again using stronger security settings where possible. The purpose of the retest was to check whether the vulnerable behaviour was still present after remediation.

I used the DVWA security levels to compare vulnerable and hardened behaviour. The retesting was limited to the same harmless XSS proof-of-concept approach used during the initial testing.

17.1 Reflected XSS Remediation and Retest

The reflected XSS vulnerability was initially confirmed on DVWA at the **Low** security level.

Initial vulnerable behaviour

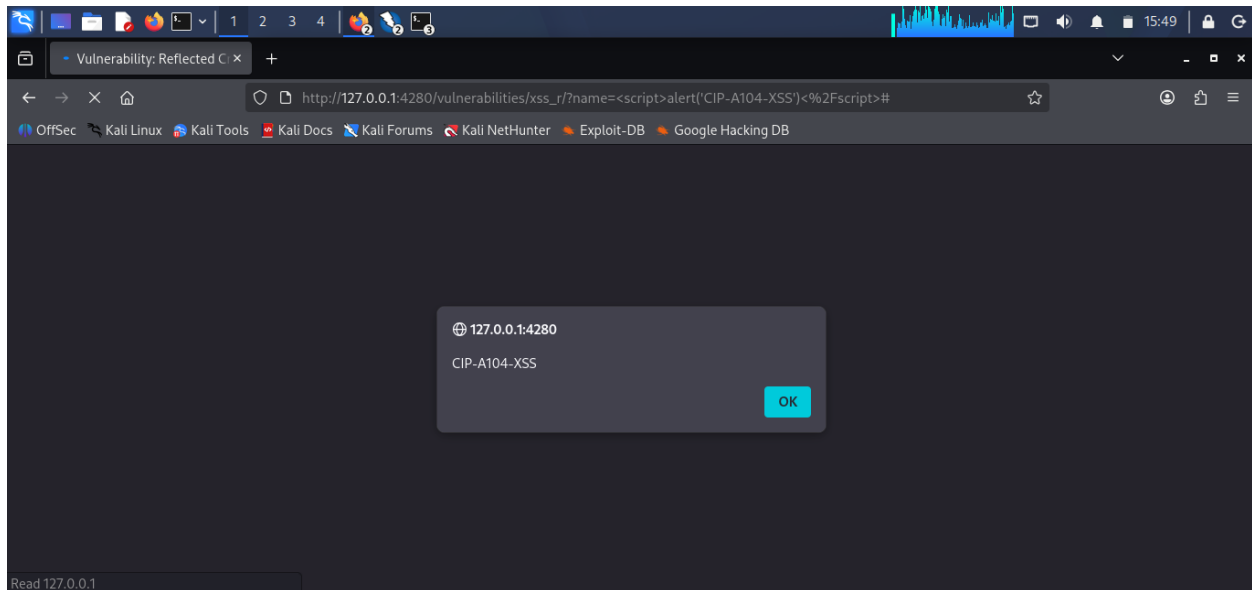
At the Low security level, I entered:

```
<script>alert('CIP-A104-XSS')</script>
```

The browser executed the JavaScript and displayed the alert:

CIP-A104-XSS

This confirmed that the input was not being properly handled before being returned to the browser.



Remediation test

I changed the DVWA security level to **Impossible** and repeated the test using the same type of harmless XSS input.

At the stronger security level, the application did not execute the JavaScript.

Instead, the input was treated as text rather than being executed as a script.

Retest result

Result: Passed / Improved

The original reflected XSS behaviour was no longer reproduced at the Impossible security level.

This showed that the stronger security controls changed the way the input was processed and prevented the test JavaScript from executing.

Residual risk

The retest only covered the specific reflected XSS behaviour tested. It does not prove that every possible XSS issue in the application has been eliminated.

Retest result

Result: Passed / Improved

The original reflected XSS behaviour was no longer reproduced at the Impossible security level.

This showed that the stronger security controls changed the way the input was processed and prevented the test JavaScript from executing.

Residual risk

The retest only covered the specific reflected XSS behaviour tested. It does not prove that every possible XSS issue in the application has been eliminated.

17.2 Stored XSS Remediation and Retest

Stored XSS was also previously tested in DVWA.

During the earlier Low-level testing, I used:

```
<script>alert('CIP-A104-STORED')</script>
```

The script executed in the first session and the payload was stored. When the application was viewed from a second browser session, the stored payload was still visible.

I did not observe a second-session alert and therefore did not claim that the script executed in the second session.

Remediation test

I changed the DVWA security level to **Impossible** and performed a new stored-input test.

The test values were:

Name: CIP-104-ST

Message: <script>alert('\CIP-A104-STORED-RETEST\')</script>

The test was intentionally harmless and was only used to determine whether JavaScript would execute.

Retest result

Result: Improved / No execution observed

No JavaScript alert appeared during the Impossible-level test.

This means that the specific stored XSS execution behaviour was not reproduced during this retest.

The important evidence here is the absence of the alert at the hardened level. I did not claim that the application was completely free of stored XSS based on this single retest.

17.3 Remediation and Retest Summary

Finding	Initial Test	Remediation/Stronger Setting	Retest Result	Status
Reflected XSS	JavaScript alert executed at DVWA Low	DVWA Impossible	Payload did not execute	Improved / Remediated for tested case
Stored XSS	Stored payload executed during earlier Low-level test	DVWA Impossible	No alert appeared during retest	Improved / Not reproduced
Session Cookie Security	HttpOnly, Secure and SameSite were not enabled	No specific remediation retest completed	No new security-attribute evidence collected	Remains open
Mutillidae Information Disclosure	Headers and HTML comment exposed information	No specific remediation retest completed	No remediation evidence collected	Remains open

18. Prioritized Final Findings and Recommendations

After reviewing ZAP results and manual testing, I ranked the findings based on their observed impact and whether I was able to confirm them.

1. **Reflected XSS – DVWA (High):** Highest priority because I successfully executed a harmless JavaScript alert in the browser. The main recommendation is to properly validate and encode user input.
2. **Weak Session Cookie Security – DVWA (Medium):** The session cookie did not have HttpOnly, Secure, or SameSite protections. These should be enabled to improve session security.
3. **Information Disclosure – Mutillidae II (Low/Medium):** Server/PHP version information and database-related information were exposed. Unnecessary technical and configuration information should be removed.
4. **Other ZAP alerts:** ZAP also reported XSS, Path Traversal, Remote File Inclusion, SQL Injection, and CSRF-related issues in Mutillidae/DVWA. These were **not manually confirmed**, so I did not report them as confirmed vulnerabilities.

19 Summary — Cleanup and Reset

I reset both applications after completing the testing.

- **DVWA:** Database reset was successful and the page showed “**Setup successful!**”
- **Mutillidae II:** Database reset was successful.
- I avoided including actual session IDs or other sensitive information in the final report.
- The reset returned both applications to a clean state and completed the assessment.

20. Activity Log

I carried out the assessment in the following order:

1. Created the Lab 5 workspace and evidence folders.
2. Started the DVWA and Mutillidae II Docker applications.
3. Launched OWASP ZAP and enabled Protected Mode.
4. Added both local applications to the ZAP scope.
5. Browsed DVWA through ZAP and recorded the site tree and passive alerts.
6. Browsed Mutillidae II through ZAP and recorded the available site information and alerts.
7. Performed the active scan against the in-scope Mutillidae II target.
8. Reviewed the ZAP alerts and separated automated findings from manually confirmed findings.
9. Manually confirmed reflected XSS in DVWA using a harmless alert payload.
10. Reviewed DVWA session-cookie security settings.
11. Reviewed Mutillidae response headers and exposed information.
12. Attempted to manually validate the Mutillidae XSS alert, but the affected page timed out.
13. Retested the DVWA XSS controls at the stronger security level.
14. Performed the Stored XSS remediation retest and observed that the test payload did not produce an alert at the stronger level.
15. Reset the DVWA database.
16. Reset the Mutillidae II database.

17. Reviewed the collected evidence and prepared the final findings, recommendations and report.

21. Evidence Register

The following evidence was collected during the assessment and used to support the findings in this report.

ID	Evidence	Application	Purpose
E01	ZAP Protected Mode and scope	DVWA / Mutillidae	Shows that testing was limited to the authorized local targets
E02	DVWA site tree and passive alerts	DVWA	Records automated passive findings
E03	Mutillidae site tree and alerts	Mutillidae II	Records discovered application paths and security alerts
E04	Mutillidae active scan results	Mutillidae II	Records alerts identified by active scanning
E05	Reflected XSS test and browser alert	DVWA	Confirms reflected XSS manually
E06	Reflected XSS retest at stronger security level	DVWA	Shows improved behavior after remediation
E07	HTTP response headers and HTML comment	Mutillidae II	Supports the information-disclosure finding
E08	Mutillidae XSS timeout message	Mutillidae II	Shows why the ZAP XSS alert was not manually confirmed
E09	Stored XSS retest at stronger security level	DVWA	Shows that the test payload did not trigger an alert
E10	DVWA database reset evidence	DVWA	Confirms cleanup and reset
E11	Mutillidae database reset evidence	Mutillidae II	Confirms cleanup and reset
E12	Activity log	Both applications	Records the assessment activities
E13	Final findings and recommendations	Both applications	Provides the final prioritized results

The evidence register helped me keep track of the information collected during the lab and connect each finding to the evidence used to support it.

22. Answers to the Analysis Questions

1. Which validated finding presents the greatest observed risk, and why?

The **Mutillidae Cross-Site Scripting (XSS) alert** was the most important alert I investigated, but I could not fully confirm it manually because the Echo Message page timed out.

Based on the findings I was able to validate, the main confirmed issue was the exposure of security weaknesses through the application's responses and configuration. However, I did not claim any finding as confirmed unless I had enough evidence to support it.

2. Which ZAP alert required the most manual interpretation?

The **Mutillidae XSS alert** required the most manual interpretation. ZAP reported the alert, but when I tried to check it manually, the affected page timed out. Therefore, I did not classify it as a confirmed vulnerability. The **Cross-Site Scripting alert in Mutillidae II** required the most manual interpretation.

ZAP reported it as a High-risk alert with Medium confidence. I tried to check the affected Echo Message page manually using a harmless test, but the page did not load and ZAP returned a timeout error.

Because I could not complete the manual test, I classified the alert as **not manually confirmed** instead of saying that the vulnerability was definitely present.

3. What is the difference between automated discovery and a confirmed finding?

Automated discovery means that ZAP has identified something that **may be a security problem** based on its scan.

A confirmed finding requires additional testing or evidence to show that the suspected vulnerability actually exists and produces the expected security impact.

For example, ZAP identified XSS, SQL Injection, Path Traversal and Remote File Inclusion in Mutillidae. I did not automatically treat all of these as confirmed vulnerabilities because I did not manually reproduce them.

4. What did the retest prove and what residual risk remains?

The retest showed that the tested XSS payload did not produce the same execution result at the stronger security level. In the Stored XSS retest, the payload did not produce an alert.

This showed that the stronger security setting improved the behaviour for the tested input. However, there is still some residual risk because the retest only covered the specific input and test case that I checked. Other application paths or vulnerabilities identified by ZAP were not fully manually validated, so further controlled testing would still be required.

23. Conclusion

This lab allowed me to perform an end-to-end security assessment of DVWA and Mutillidae II using OWASP ZAP and manual testing.

I identified several security issues and separated the automated ZAP alerts from the findings that I could actually confirm. The main confirmed finding was reflected XSS in DVWA. I also identified weak session-cookie security in DVWA and information disclosure in Mutillidae II.

I also performed remediation retesting and observed improved behavior at the stronger security level. Finally, I reset both applications and prepared the evidence and findings for the final report.

Overall, the assessment showed me the importance of combining automated scanning with manual validation instead of treating every scanner alert as a confirmed vulnerability.

